

COMP 5111 (2026Spring)

Assignment 1 Introduction & Soot Tutorial

(Preview Version)

TA: Songqiang CHEN

Contact TAs

Name: CHEN, Songqiang
LU, Weiqi

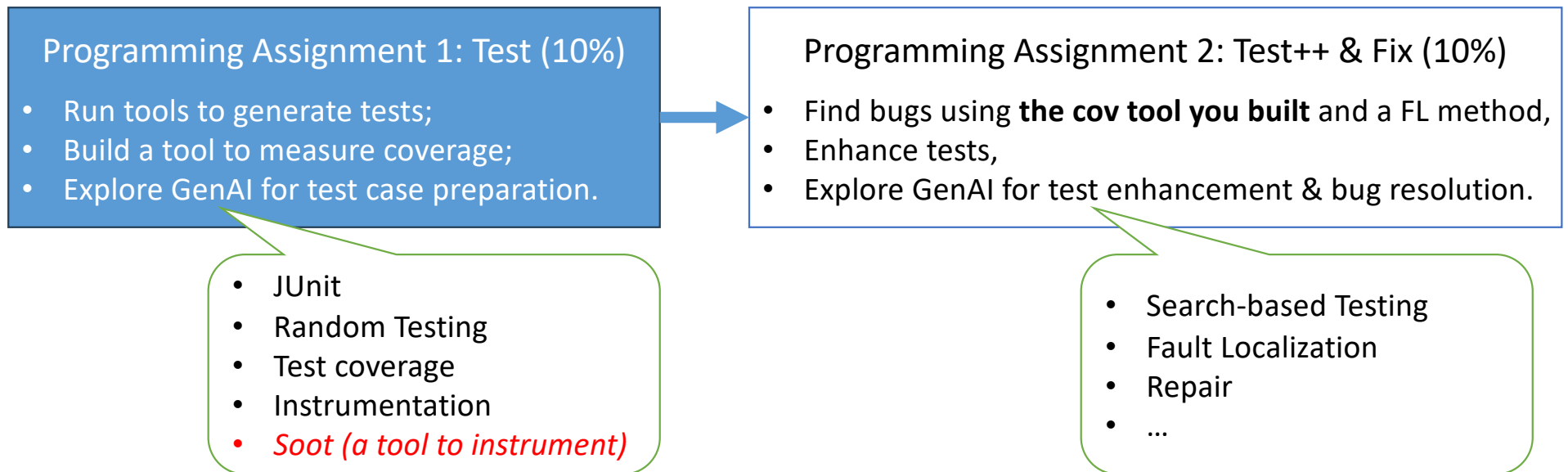
Email: csta5111@cse.ust.hk

Office Hour: Rm 3663 (via Rm3661)
Tue & Thu 11:50 - 12:20

Zoom: Please send us an email to
schedule a date and timeslot.

Programming Assignments of COMP5111 25S (20%)

To test and fix a Java program in "Subject.java" (Subject Class, Class Under Test, CUT).



Programming Assignment Environment

- **OS: Linux / MacOS / Windows**
 - Linux is highly recommended.
 - You can use the machine at CS lab.
- **JDK 11**
- **Eclipse 2025.12**
- **JUnit 4.12**, with hamcrest-1.3
- **Randoop 4.3.1**
- **Soot 4.2.1**

(May not be the latest, but should be stable and well compatible.)

Programming Assignment 1 Overview (10 pts)

- **Tasks:**

- Task 1: Run Tools to Generate Test Cases and Measure Code Coverage (25%)
 - Task 1.1: To learn how to use **Randoop** for unit test generation
 - Task 1.2: To learn how to use **EclEmma** for code coverage measurement.
- Task 2: Line coverage measurement using **Soot** (40%) [Soot Tutorial]
- Task 3: Exploring the usefulness of **Gen AI** for specification-based testing (35%)

- **Submission:** Canvas, by 15 March 2026 (23:55)

Detailed Specifications: <https://github.com/CastleLab/COMP5111-2026Spring-Assignments-Students>

Task 1 (25%): Generate Test Cases & Measure Code Coverage

- Basically, task 1 only asks you to **run existing software** to generate tests and measure coverage.
- The most you need to do is following our instructions in our repository. Some requirements may need your exploration of certain advanced parameters.
- Most steps are quite easy. Hints:
 - Make sure that you **correctly install** any related software and **set appropriate** environment variables and parameter values.
 - Some students get error: *Class Not Found* – Check your Java classpath setup.

Task 1.1: Generate Tests with Randoop

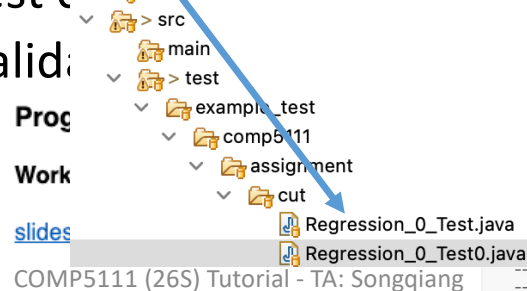
- Objective:

- Use Randoop to generate **FIVE regression test suites** for the program under test (named Subject.java).

- Tutorials: Available in our repository.
An example script is given in the example project.

- Steps:

- Run Randoop to yield test cases
- Run the test cases for validation



Preview Version.

Random Testing with Randoop

Introduction

In this tutorial, you will learn how to use the random testing tool Randoop. Randoop is an automatic unit test generator for Java programs. It generates two types of test suites:

- Error-revealing tests that indicates errors in the Java code.
- Regression tests that capture current behavior of the program.

Steps

You can

1. Install Randoop

2. Generate test suites

3. Run the test suites

4. Analyze the results

5. Repeat the process

6. Submit the results

7. Review the results

8. Generate a report

9. End of the process

Task 1.1: Learn how to use Randoop for unit test generation.

Randoop is a tool for JUnit test generation. A short usage guide for Randoop is available [here](#), covering how to generate JUnit tests using Randoop and run them in Eclipse.

Requirements:

- You need to use Randoop to generate five different test suites for the given program.
- Each test suite must achieve at least 50% line coverage and at least 40% branch coverage in EcJemma. You may need to explore proper Randoop parameters to achieve the coverage requirements.
- The five test suites should be different from each other. Randoop parameter `--randomseed` may be helpful.
- The package name of the generated test suites should be `comp5111.assignment.cut`. Randoop parameter `--junit-package-name` may be helpful.
- The class name of the generated regression test suite should be `Regression_{i}_Test`, `i` in `[0..4]`. Randoop parameter `--regression-test-basename` may be helpful.
- Please only submit and measure the coverage of the regression tests generated by Randoop. Do not submit and measure coverage of the error-revealing tests.
- The five regression test suites should be put in `src/test/randoop<i>` for the `<i>`th test suite, where `0<=i<=4`. Randoop parameter `--junit-output-dir` may be helpful.

```
1 package comp5111
2
3 import org.junit.*
4
5 @FixMethodOrder
6 public class Regression_0_Test {
7
8     public static void main(String[] args) {
9
10        // Requirements:
11
12        // 1. You need to use Randoop to generate five different test suites for the given program.
13        // 2. Each test suite must achieve at least 50% line coverage and at least 40% branch coverage in EcJemma. You
14        //    may need to explore proper Randoop parameters to achieve the coverage requirements.
15        // 3. The five test suites should be different from each other. Randoop parameter --randomseed may be helpful.
16        // 4. The package name of the generated test suites should be comp5111.assignment.cut. Randoop parameter --
17        //    junit-package-name may be helpful.
18        // 5. The class name of the generated regression test suite should be Regression_{i}_Test, i in [0..4]. Randoop
19        //    parameter --regression-test-basename may be helpful.
20        // 6. Please only submit and measure the coverage of the regression tests generated by Randoop. Do not submit
21        //    and measure coverage of the error-revealing tests.
22        // 7. The five regression test suites should be put in src/test/randoop<i> for the <i>th test suite, where
23        //    0<=i<=4. Randoop parameter --junit-output-dir may be helpful.
24
25        boolean1 = comp5111.assignment.cut.Subject.NumberTasks.isParsable("");
26        org.junit.Assert.assertTrue("'" + boolean1 + "' != '" + false + "'", boolean1 == false);
27
28        @Test
29        public void test04() throws Throwable {
30            if (debug)
31                System.out.format("%n%s%n", "Regression_0_Test0.test04");
32            java.lang.String str1 = comp5111.assignment.cut.Subject.StringTasks.chop("hi!");
33            org.junit.Assert.assertEquals("'" + str1 + "' != '" + "hi" + "'", str1, "hi!");
34        }
35
36        @Test
37        public void test05() throws Throwable {
38            if (debug)
39                System.out.format("%n%s%n", "Regression_0_Test0.test05");
40            java.lang.String str1 = comp5111.assignment.cut.Subject.FilenameTasks.getName("hi!");
41            org.junit.Assert.assertEquals("'" + str1 + "' != '" + "hi" + "'", str1, "hi!");
42        }
43    }
44}
```

Task 1.2: Measuring Test Coverage using EclEmma (in Eclipse)

- Objective:
 - Use EclEmma to measure the coverage of generated test cases.
- Tutorials: Avail in our repository.
- Steps:
 - Install **EclEmma / Eclipse**.
 - Create/Import the Java project.
 - Run the unit test cases.
 - Read and check the report.

Example screenshot (some notes):

Test coverage measurement using EclEmma in Eclipse

Introduction

In this tutorial, you will learn how to use the toolkit EclEmma in Eclipse to measure test coverage. (Please note that some screenshots and demo video are materials of last year's course.)

Steps

You can follow these steps to build the JAVA project on Eclipse, then use it to generate test coverage.

Step 0: Install Eclipse 2023-12 and EclEmma

You can install the Eclipse 2023-12 following the [official website](#). If you have already installed Eclipse, you can upgrade your Eclipse through [this link](#).

NOTE: EclEmma is already in Eclipse 2023-12 by default. You may skip this step.

Step 1: Another common issue: Coverage = 0?

As you have seen, the coverage is 0. This is because the test suite is not selected correctly. The correct test suite is **JUnit 4**.

Step 1-1: Coverage As

Click on **Coverage As** and select **JUnit 4** from the dropdown menu.

Regression_6_Test (1) 2026年2月8日 下午3:44:03

Element	Coverage	Covered Branches	Missed Branches	Total Branches
comp5111.assignment.cut	20.5 %	218	848	1,066
Subject.java	40.9 %	218	315	533
Subject		0	0	0
DateTimeAlgorithms	74.1 %	103	36	139
GameConfiguration	4.5 %	5	107	112
GamePlace	37.5 %	6	10	16
GamePlayer	66.7 %	4	2	6
StringAlgorithms	38.5 %	100	160	260

show the name of test suite.

show the coverage of CUT and all inner classes. (NOT test suite!)

Task 1 (25%): Generate Test Cases & Measure Code Coverage (cont.)

- **Submissions:**

- **Task 1.1:** 5 regression test suites generated by you.

- **Task 1.2:** 10 screenshots.

For each regression test suite, one for line coverage and one for branch coverage.

- **Requirements:** -- *failure in these points cannot earn scores.*

- **1.1:** Submit the **regression tests** generated by Randoop, but not the **error-revealing tests**.

- **1.2:** Each test suite must achieve **>= 50% line coverage** & **>= 40% branch coverage** (reported by EclEmma)

- **2.1:** The coverage is measured on the Subject class but not the tests (all previous notes).

- **2.2:** The coverage shown in the screenshot should be measured by Eclipse, not other IDEs.

- **More Details on Format Requirement, Grading Scheme, etc.:**

(All are concretely explained in the detailed instruction in our repository.)

Task 2 (40%): Measuring Test Coverage with Soot

- **Objective:** Develop a coverage measurement tool with Soot!
- Sounds a bit hard?
It is easy - once you understand: the idea of instrumentation, and how to implement instrumentation using Soot.

Task 2 (40%) – Background Knowledge

• What to measure?

Executed Tests

```
@Test
public void test1() {Subject.StringAlgorithms.startsWithIgnoreCase("1", "12");}
@Test
public void test2() {Subject.StringAlgorithms.startsWithIgnoreCase("12", "1");}
```

Covered Lines (with statements) in CUT

```
20 public static boolean startsWithIgnoreCase(String str, String prefix) {
21     if (str == null || prefix == null) {
22         return false;
23     }
24     if (str.length() < prefix.length()) {
25         return false;
26     }
27     if (str.startsWith(prefix)) {
28         return true;
29     }
30     String lcStr = str.substring(0, prefix.length()).toLowerCase();
31     String lcPrefix = prefix.toLowerCase();
32     return lcStr.equals(lcPrefix);
33 }
34
```

Problems Javadoc Declaration Search Console Progress Terminal Coverage x

Sample (2026年2月8日 下午4:10:16)

Element	Coverage	Covered Lines	Missed Lines	Total Lines
startsWithIgnoreCase(String, String)	55.6 %	5	4	

(P)review of the related coverage: (a rough recap :)

- Line Coverage: the **% lines that have been executed** among all in the class under test **on a set of specified test cases.**

$$\text{Coverage} = \frac{\# \text{executed lines}}{\# \text{total lines}}$$

- In general, not consider the “empty” lines without statements.
- Why do we care about it?
– A bug will NOT be detected unless it is "reached". (?)

Task 2 (40%) – Background Knowledge (cont.)

- **Measure on what code?**

We use **Jimple 3-Address Code**, the Java intermediate code used by Soot, in our PA.

- Instead of directly handling *Java src/bin code*, in our assignment, we **process the *Jimple 3AC statements*** and **count the Java source lines associated with them**.

Java:

```
...  
(1) for (i = 0; i < 10; ++i){  
(2)     b[i] = i * i;  
(3) }  
...
```



```
    t1 := 0           (1) ; initialize i  
L1:  if t1 >= 10 goto L2 (1) ; conditional jump  
    t2 := t1 * t1      (2) ; square of i  
    t3 := t1 * 4       (2) ; word-align address  
    t4 := b + t3       (2) ; address to store i*i  
    *t4 := t2          (2) ; store through pointer  
    t1 := t1 + 1       (1) ; increase i  
    goto L1            (1) ; repeat loop  
  
L2:
```

Task 2 (40%) – Background Knowledge (cont.)

- **How to measure?** (Methodology, not implementation)

Instrument: Insert some statements into the target program, in order to conduct additional monitors for program, while not changing the original code behaviors.

- To answer “how many lines are executed”, we can edit the code so that:
 - Before running CUT, **we insert some code** to let machine auto record which stmt & line have been executed, and output the record at the end of execution.

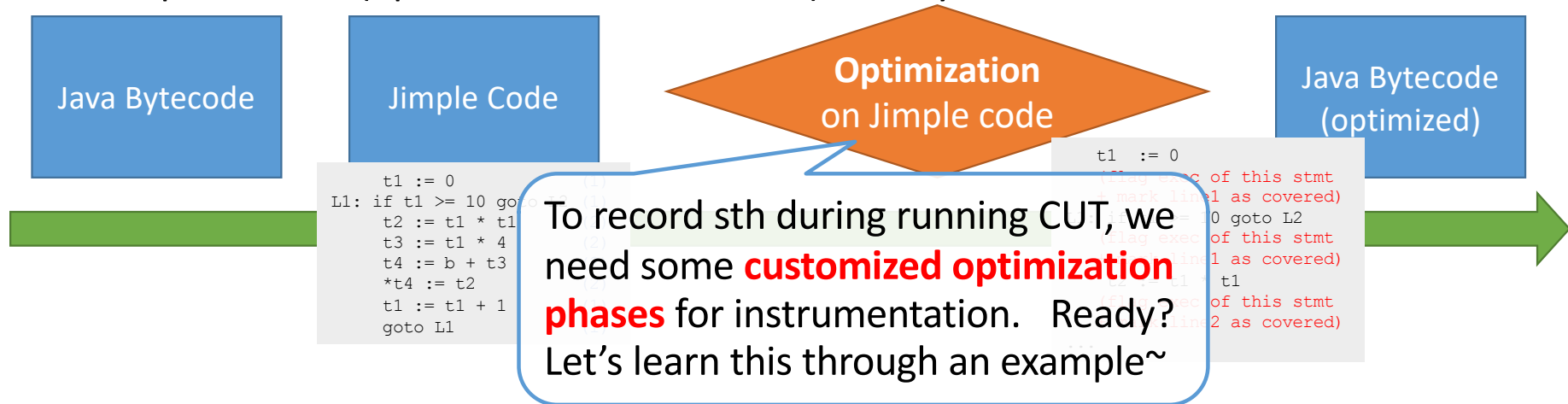
```
t1 := 0           (1)
L1: if t1 >= 10 goto L2 (1)
    t2 := t1 * t1   (2)
    t3 := t1 * 4     (2)
    t4 := b + t3     (2)
    *t4 := t2        (2)
    t1 := t1 + 1     (1)
    goto L1          (1)
L2:
```

```
t1 := 0
(flag exec of this stmt
+ mark line1 as covered)
L1: if t1 >= 10 goto L2
(flag exec of this stmt
+ mark line1 as covered)
    t2 := t1 * t1
(flag exec of this stmt
+ mark line2 as covered)
...
```

- Then, during running CUT, the CUT with inserted code **remains the original functionality**, but it **will further** automatically **record** and **output** the runtime information.

Task 2 (40%) – Soot

- We use a tool called **Soot** to instrument program automatically.
 - A tool for Optimization & **Instrumentation** (automatically edit the code when traversing and analyzing the code.)
 - Input: **Java Bytecode** (what we use) / Java Source Code.
 - Intermediate Representation: **Jimple** (what we use) / Baf / Shimple / Grimp.
 - Output: Edited (optimized / instrumented) Java Bytecode.



Task 2 (40%) – Example: Count Executed Returns using Soot

- **HOW?** – A Demo to Illustrate the **Recording & Output** Implementation.
(download the project, and take notes)

Coursework

Programming Assignments

Workshop: Assignment 1 Introduction

slides [soot example project](#)

- **Task:** Count how many times the return statement in static / instance methods in CUT are invoked by a test suite, respectively.

```
public class Subject {  
    public static int staticMethod_Add(int x, int y) { return x + y; }  
    public static double staticMethod_Add(double x, double y) { return x + y; }  
    public static int staticMethod_Mul(int x, int y) { return x * y; }  
    public static double staticMethod_Mul(double x, double y) { return x * y; }  
    public int instanceMethod_Add(int x, int y) { return x + y; }  
    public double instanceMethod_Add(double x, double y) { return x + y; }  
    public int instanceMethod_Mul(int x, int y) { return x * y; }  
    public double instanceMethod_Mul(double x, double y) { return x * y; }  
}  
  
void main() {  
    new Subject().staticMethod_Add(1, 2);  
    new Subject().instanceMethod_Add(1, 2);  
    new Subject().staticMethod_Add(1, 2);  
    // 2, 1  
}
```

- First of all, consider **insert what code to where with what utilities?**
 - Sol: **Insert** a statement to **increase a counter before each return stmt.**
- Then, implement to realize the transformation.

Task 2 (40%) – Example Use of Soot

Implement: insert **what** code to **where**, with **what utilities**

> Insert a statement to **increase a counter before each return stmt.**

- Implement another set of Java source codes as our tool.
- These codes consist our tool. They are NOT a part of the program to test (Subject.java).
- **What do we need? Three components:**
 - **A counter class** maintaining several variables / functions to record sth (e.g., a **var** for how many times a static return stmt is invoked; a **func** to increase this var; ...)
 - **An instrumentation class (a customized transformation)** that defines what to do when traversing a code snippet (e.g., to call the **func** to increase the **var** before each return)
 - **An entry point for the whole tool** to setup soot and start the transformation.

Task 2 (40%) – Example Use of Soot – Our Own Phase – Counter

Implement: insert **what** code to **where**, with **what utilities**

- The helper class for recording data: **Counter**

```
1 package castle.comp5111.example;
2
3 public class Counter {
4     private static int numStaticInvocations = 0;
5     private static int numInstanceInvocations = 0;
6
7     public static void addStaticInvocation(int n) { numStaticInvocations += n; }
8
9     public static void addInstanceInvocation(int n) {
10         numInstanceInvocations += n;
11     }
12
13
14
15     public static int getNumInstanceInvocations() { return numInstanceInvocations; }
16
17     public static int getNumStaticInvocations() { return numStaticInvocations; }
18
19
20
21
22 }
```

Counter variables to record what we count

How to realize this?

The function to call (invocation inserted to CUT) to increase private counter variables.

The function to call after executing the instrumented CUT to print counting results.

```
package castle.comp5111.example;
/* This class is the subject in our tutorial of soot.
 * We first generate tests using randop.
 * Then, we will use soot to instrument this class and count the total number of static/instance methods are called.
 * In tests.
 */
public class Subject {
    public static int staticMethod_Add(int x, int y) { return x + y; }
    public static double staticMethod_Add(double x, double y) { return x + y; }
    public static int staticMethod_Mul(int x, int y) { return x * y; }
    public static double staticMethod_Mul(double x, double y) { return x * y; }
    public int instanceMethod_Add(int x, int y) { return x + y; }
    public double instanceMethod_Add(double x, double y) { return x + y; }
    public int instanceMethod_Mul(int x, int y) { return x * y; }
    public double instanceMethod_Mul(double x, double y) { return x * y; }
}

void main() {
    new Subject().staticMethod_Add(1, 2);
    new Subject().instanceMethod_Add(1, 2);
    new Subject().staticMethod_Add(1, 2);
    // 2, 1
    System.out.println(...);
}
```

Task 2 (40%) – Example Use of Soot – Our Own Phase – Transformer

Implement: insert **what** code to **where**, with **what recorder**

- How to auto insert invocation to the var-incr function into CUT?
 - A customized **BodyTransformer** to insert **invocation statements** **before each return statement**.

```
public class Instrumenter extends BodyTransformer {  
    /* some internal fields */  
    static SootClass counterClass;  
    static SootMethod addStaticInvocationMethod,  
    addInstanceInvocationMethod;  
    /*  
     * internalTransform goes through a method body and inserts counter  
     * instructions before method returns  
     */  
    @Override  
    protected void internalTransform(Body body,  
        // body's method  
        SootMethod method = body.getMethod();  
    // we dont instrument constructor (<init>  
    if (method.isConstructor() || method.is  
        return;  
    }  
    // debugging  
    System.out.println("instrumenting method: " + method.getSignature());  
    // get body's unit as a chain  
    Chain<Unit> units = body.getUnits();  
    // get a snapshot iterator of the unit since we are going to  
    // mutate the chain when iterating over it.  
    //  
    Iterator<?> stmtIt = units.snapshotIterator();  
}
```

=> Optimization phase registered to soot.

Java Bytecode → Jimple Code → Optimization on Jimple code → Java Bytecode (optimized)

Key method: **internalTransform**
It will be executed when each method in the CUT is analyzed.

```
// typical while loop for iterating over each statement  
while (stmtIt.hasNext()) {  
    // cast back to a statement.  
    Stmt stmt = (Stmt) stmtIt.next();  
    // there are many kinds of statements, here we are only  
    // interested in return statements  
    // NOTE: there are two kinds of return statements, with or without return value  
    if (stmt instanceof ReturnStmt || stmt instanceof ReturnVoidStmt) {  
        // now we reach the real instruction  
        // call Chain.insertBefore() to insert instructions  
        //  
        // 1. first, make a new invoke expression  
        InvokeExpr incExpr = null;  
        if (method.isStatic()) {  
            // if current method is static, we add static method invocation counter  
            incExpr = Jimple.v().newStaticInvokeExpr(  
                addStaticInvocationMethod.makeRef(), IntConstant.v(1));  
        } else {  
            // if current method is instance method, we add instance method invocation counter  
            incExpr = Jimple.v().newStaticInvokeExpr(  
                addInstanceInvocationMethod.makeRef(), IntConstant.v(1));  
        }  
        // 2. then, make a invoke statement  
        Stmt incStmt = Jimple.v().newInvokeStmt(incExpr);  
        // 3. insert new statement into the chain, before return statement  
        // (we are mutating the unit chain).  
        units.insertBefore(incStmt, stmt);  
    }  
}
```

How to identify the place to insert?
-- Find the "ReturnStmt"

How to make an invocation to the funcs?
-- Soot API 'newStaticInvokeExpr'.

How to insert? – Soot API 'newInvokeStmt' and 'insertBefore'.

Task 2 (40%) – Example Use of Soot – Tool Entrypoint

- **An entry point for the whole tool to setup Soot and start the transformation.**
(you may further understand it after tutorial by yourself)

- Overall Procedure:
Compile => Instrument => Run CUT on Tests => Result

```
public class EntryPoint {  
    public static void main(String[] args) {  
        instrumentWithSoot();  
        // after instrument, we run Junit tests  
        runJunitTests();  
        // after junit test running, we have already get the counting in the Counter class  
        System.out.println("Invocation to static methods: " + Counter.getNumStaticInvocations());  
        System.out.println("Invocation to instance methods: " + Counter.getNumInstanceInvocations());  
    }  
}
```

- Add our phase *Instrumenter* into Soot
- Our customized phase will be executed by:

```
private static void instrumentWithSoot() {  
    // the path to the compiled Subject class file  
    String classUnderTestPath = "./raw-classes";  
    String targetPath = "./target/classes";  
  
    String classPathSeparator = ":";  
    if (System.getProperty("os.name").toLowerCase().contains("win")) {  
        classPathSeparator = ";";  
    }  
  
    /*Set the soot-classpath to include the helper class and class to analyze*/  
    Options.v().set_soot_classpath(Scene.v().defaultClassPath() + classPathSeparator + targetPath +  
  
    // we set the soot output dir to target/classes so that the instrumented class can override the  
    Options.v().set_output_dir(targetPath);  
  
    // retain line numbers  
    Options.v().set_keep_line_number(true);  
    // retain the original variable names  
    Options.v().setPhaseOption("jb", "use-original-names:true");  
  
    /* add a phase to transformer pack by call Pack.add */  
    Pack jtp = PackManager.v().getPack("jtp");  
  
    Instrumenter instrumenter = new Instrumenter();  
    jtp.add(new Transform("jtp.instrumenter", instrumenter));  
  
    String classUnderTest = "castle.comp5111.example.Subject";  
    // pass arguments to soot  
    soot.Main.main(new String[]{classUnderTest}); // added phases will be executed in this method  
}
```

Task 2 (40%) – Example Use of Soot (cont.)

Check what a Jimple code looks like: - may help you understand & debug (original vs edited).

Foo.java

```
1 public class Foo {
2
3     static int factorial (int x){
4         int result = 1;
5         int i = 2;
6         while (i <= x) {
7             result *= i;
8             i++;
9         }
10        return result;
11    }
12 }
13 }
```

Foo.jimple

```
1 public class Foo extends java.lang.Object
2 {
3
4     public void <init>()
5     {
6         Foo r0;
7
8         r0 := @this: Foo;
9
10        specialinvoke r0.<java.lang.Object: void <init>()>();
11
12        return;
13    }
14
15    static int factorial(int)
16    {
17        int i0, i1, i2;
18
19        i0 := @parameter0: int;
20
21        i1 = 1;
22
23        i2 = 2;
24
25        label1:
26        if i2 > i0 goto label2;
27
28        i1 = i1 * i2;
29
30        i2 = i2 + 1;
31
32        goto label1;
33
34        label2:
35        return i1;
36    }
37 }
```

Commands for easy understanding

java soot.Main -f jimple Foo

Same as above, but produce Jimple in sootOutput/Foo.jimple.

java soot.Main Foo

Process Foo.class in the current directory and produce a new class file in sootOutput/Foo.class.

Task 2 (40%) – Requirements: Output Format & Runner Scripts

- Req 1: coverage report, including **at least** the following information:
 - The line coverage for CUT (*final percentage + raw counts of cov/tot lines*), *incl. inner classes*.
 - The coverage of each statement/branch (format: *Line No, covered or missed*).
 - Encouraged: the statements in each line (and their coverage).

comp5111.assignment.cut.Regression_6_Test - Line Coverage

Overview:

Overall:

total: (*omitted*) covered: (*omitted*) percentage: 55.2%

comp5111.assignment.cut.Subject\$DateTimeAlgorithms

total: 159 covered: 101 percentage: 63.5%

...(omitted. please list the coverage of all inner classes)...

Details:

comp5111.assignment.cut.Subject\$DateTimeAlgorithms

int daysBetweenDateStrings(java.lang.String,java.lang.String,

Covered Line 687 (*you are encouraged to print Jimple stmts in this line:* parts1 = virtualinvoke dateStr1.<java.lang.String: java.lang.String[] split(java.lang.String)>("-"))

Missing Line 691 (*you are encouraged to print Jimple stmts in this line:* \$stack14 = parts1[0]; y1 = staticinvoke <java.lang.Integer: int parseInt(java.lang.String)>(\$stack14))

...(omitted)...

Preview Version.

Task 2: Line coverage measurement using Soot (40%)

Soot is a framework for analyzing and transforming Java programs. We prepared a short introduction about Soot [here](#). Answers to some common questions are available [here](#).

For Task 2 you may refer to an [example test suite](#) along with its [line coverage](#) collected by our implementation

Req 2: Prepare a one-click script + instruction to use your tool.

```
for (( i = 0; i < 1; i++ )); do
```

```
arguments="$i comp5111.assignment.cut.Regression_"$ti"_Test "
```

```
arguments=$arguments'comp5111.assignment.cut.Subject comp5111.assignment.cut.Subject$StringAlgorithms  
comp5111.assignment.cut.Subject$DateTimeAlgorithms (omit some classes here. please complete.) '
```

```
mvn clean compile exec:java -Dexec.mainClass="comp5111.assignment.Assignment1" -Dexec.args="$arguments"  
done
```

(Example codes here are for reference. They may be incomplete and contain minor errors.)

Task 2 (40%): Measuring Test Coverage with Soot (cont.)

Kind Tips – How to start?

- **Start from the example project**
 - Understand the **code & logic** about **traversing statements** and **recording info**. Your assignment is ***quite similar*** to the example project.
 - **Example Task (counting executed return stmts)**
 - Insert the **statements to increase counter** **for each return statement**.
 - **Task 2 (line coverage)**
 - Insert the **code to record stmts & line nos** for **each target Jimple statement**.

Figure out what to record and how to implement them by yourself.

Task 2 (40%): Measuring Test Coverage with Soot (cont.)

Other Tips

- **You can crosscheck your Soot-based results with not only TA-line.txt but also EcJemma. However, you may find the number of stmts counted by Soot is different from by EcJemma.**
 - **Possible**, because EcJemma and Soot use different intermediate representation. You may only refer to the EcJemma stmt coverage **for reference** – but they are usually **close**.
- **Record the coverage of statement as well. It is helpful for second programming assignment. A major task in PA2 is also about calculating something based on instrumentation records.**
- **Others:**
 - Go through the soot document to learn **other useful APIs** besides the ones in example project.
 - Read **FAQ** in our repository first; contact with TAs if still serious issues (but TAs won't help debug!).
 - Two Materials: <https://www.brics.dk/SootGuide/> , <https://noidsirius.medium.com/a-beginners-guide-to-static-program-analysis-using-soot-5aee14a878d>

Task 3 (35%): Explore the Use of GenAI for Test Generation

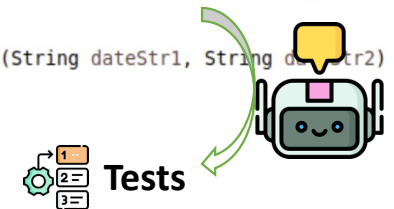
- Requirements: (Please refer to our very detailed instructions)

- The core topic:** Generate tests based on the **specification** (docstring, no code) of functions.

- Four subtasks.

- Test Generation Based on Current Specifications (10%):** Design effective prompts to make the LLM generate JUnit tests for 10-20 functions based on only the function's docstring and signature.
- Analysis of Limitations of Current Specifications for Test Generation (10%):** Identify limitations of these tests in coverage, correctness, etc., analyze what issues are caused by the current docstring's low quality (e.g., missing certain important details, ambiguous descriptions, etc.)
- Suggestions for Specification Improvement (5%):** Propose 2-4 concrete ways to mitigate docstring issues.
- Justify Effectiveness of the Enhanced Specifications (10%):** Enhance the specifications of some functions based on your suggestions, and demonstrate its effectiveness with the improvement in test case quality.

```
/**
 * Returns the number of days between two date strings in "yyyy-MM-dd" format.
 *
 * @param dateStr1 the first date string
 * @param dateStr2 the second date string
 * @return the number of days between the two dates, or Integer.MIN_VALUE if
 *         invalid
 */
public static int daysBetweenDateStrings(String dateStr1, String dateStr2) {
```



Task 3 (35%): Explore the Use of GenAI for Test Generation (cont.)

- Write a report to discuss findings in these subtasks. Length (soft limits)
 - Test Generation Based on **Current** Specifications (10%). – 100-250 words.
 - Analysis of **Limitations** of Current Specifications for Test Generation (10%). – 200-350 words.
 - **Suggestions** for Specification Improvement (5%). – 100-250 words.
 - Justify **Effectiveness of the Enhanced** Specifications (10%). – 200-350 words.
- **Note:** Keep your report informative and concise. Concrete examples, intuitive visualizations (figures), and quantitative measurements are expected to support your analysis.
- We encourage the use of HKUST hkgai-v1-series models free on HKUST OpenWebUI platform, but other publicly available LLMs are also allowed.
- **Submission:** A PDF, with the discussions in each subtask separated in independent sections.

Submission:

- **Submission Requirement:**

- Submit via **Canvas**.
- Please check our course website & repo for the very detailed specification.
 - all info mentioned in this tutorial, as well as the detailed submission format requirement, hint and guidance, FAQs, etc.

- **Deadline:** 23:55, 15th March 2026

Do it as early as possible to ensure enough debug time & Try your best.

<Reading assignment time slot booking!>